

Lecture 4: Linear Models for Classification (Part 2)

Recap: Linear model

- In dimension D , we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \dots + w^{(D)}x^{(D)} = \mathbf{w}^T \begin{bmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{bmatrix}$$

- In short, we can write

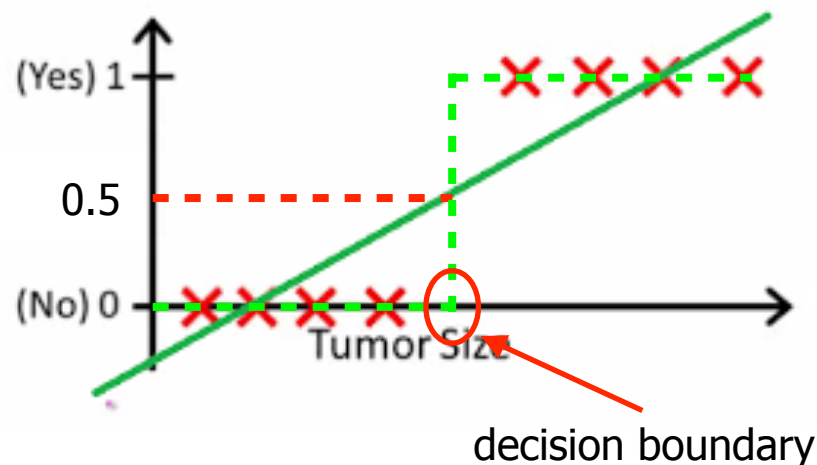
$$y = \mathbf{w}^T \mathbf{x}$$

with $\mathbf{x} \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$

Recap: Binary classification as regression

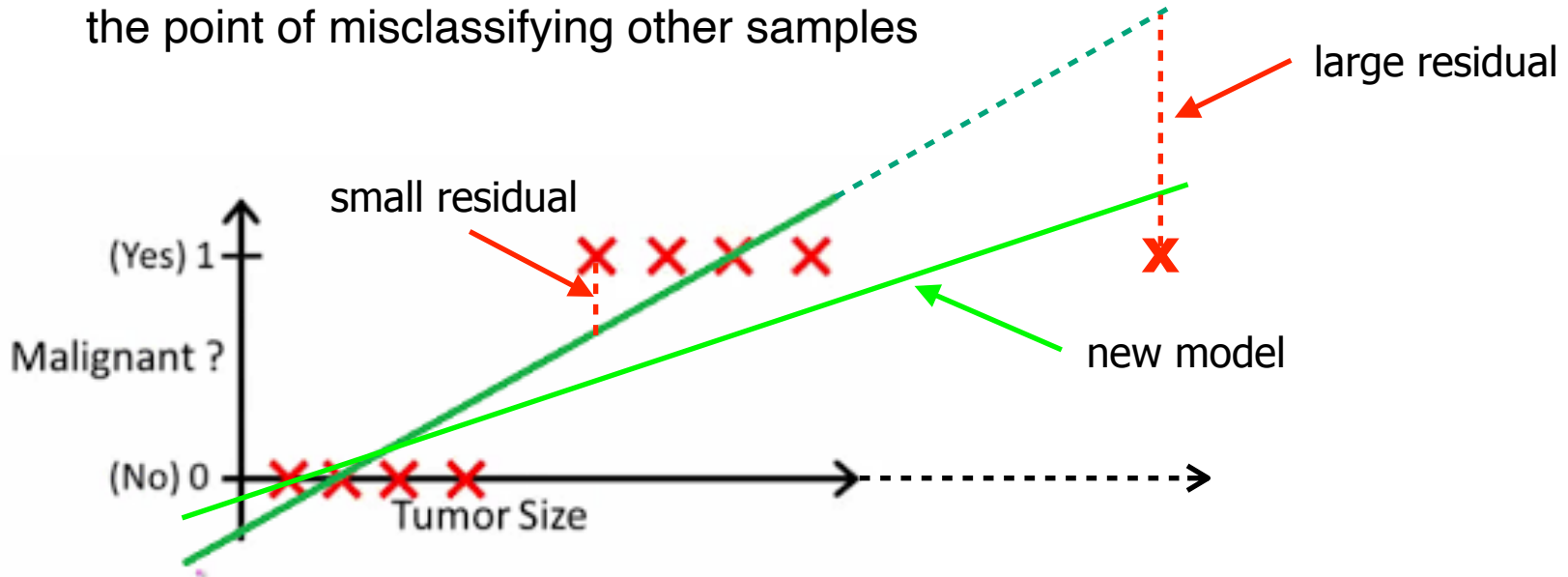
- Example (from Andrew Ng's course): Classify a tumor as benign vs malignant
 - 1D input (tumor size), 1D output (malignant vs benign)
 - Using a linear model, we can predict the label as $\hat{y} = w^{(0)} + w^{(1)}x$

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$



Recap: Failures

- Back to the tumor prediction example
 - The residuals for “normal” samples are quite small
 - However, a sample with a much larger tumor size (with true label 1) would have a much larger residual
 - Training with such a sample would strongly affect the model parameters, to the point of misclassifying other samples

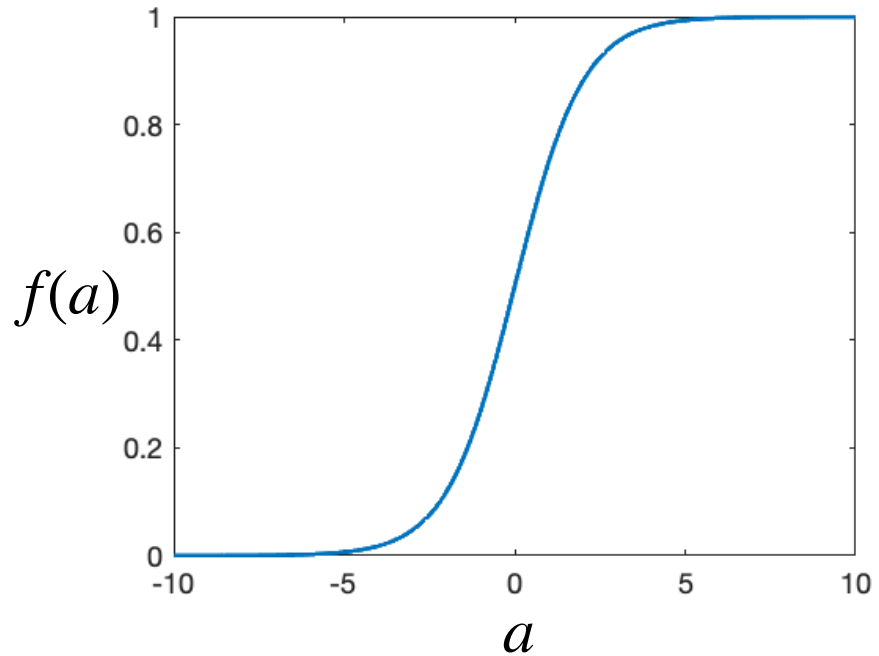


- This is because least-square classification fits a line to non-linear data

Recap: Adding non-linearity

- To model a classification problem, one can use a smooth approximation of the step function, such as the *logistic sigmoid function*

$$f(a) = \frac{1}{1 + \exp(-a)}$$



Recap: Logistic regression

- To better model the underlying binary classification problem, we can pass the output of the linear model through a logistic function

$$\hat{y}(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

- Such a prediction can be interpreted as the probability that $\mathbf{x}$ belongs to the positive class (or as a score for the positive class)
- The probability to belong to the negative class (or score for the negative class) is then computed as $1 - \hat{y}(\mathbf{x})$

Recap: Logistic regression: Training

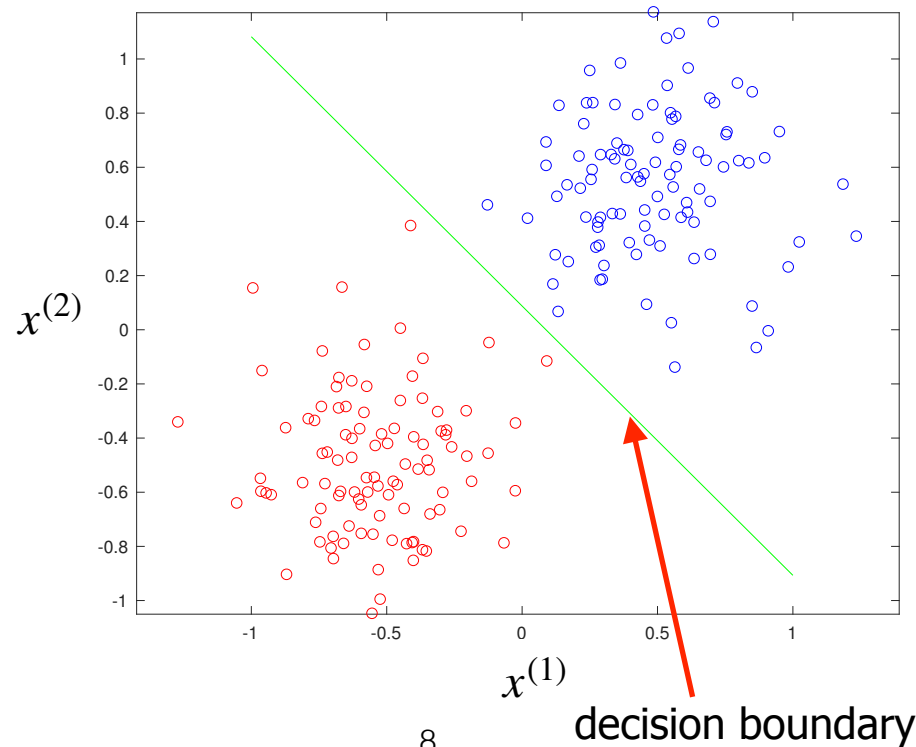
- With binary ground-truth labels y_i , training the model is done by minimizing the cross-entropy

$$\begin{aligned} R(\mathbf{w}) &= - \sum_{i \in \text{positive samples}} \ln(\hat{y}_i(\mathbf{w})) - \sum_{i \in \text{negative samples}} \ln(1 - \hat{y}_i(\mathbf{w})) \\ &= - \sum_{i \in \text{positivesamples}} \ln \sigma(\mathbf{w}^T \mathbf{x}_i) - \sum_{i \in \text{negativesamples}} \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \end{aligned}$$

- This is a convex function of $\mathbf{w}$, which can be minimized via gradient descent.

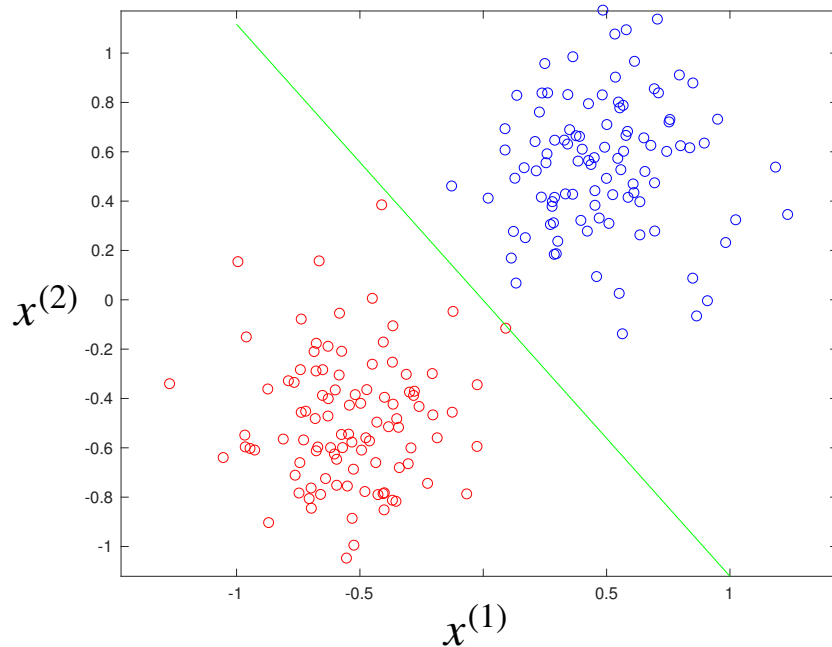
Recap: Decision boundary

- In the binary case, the region in the input space where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 2D inputs, it is a line

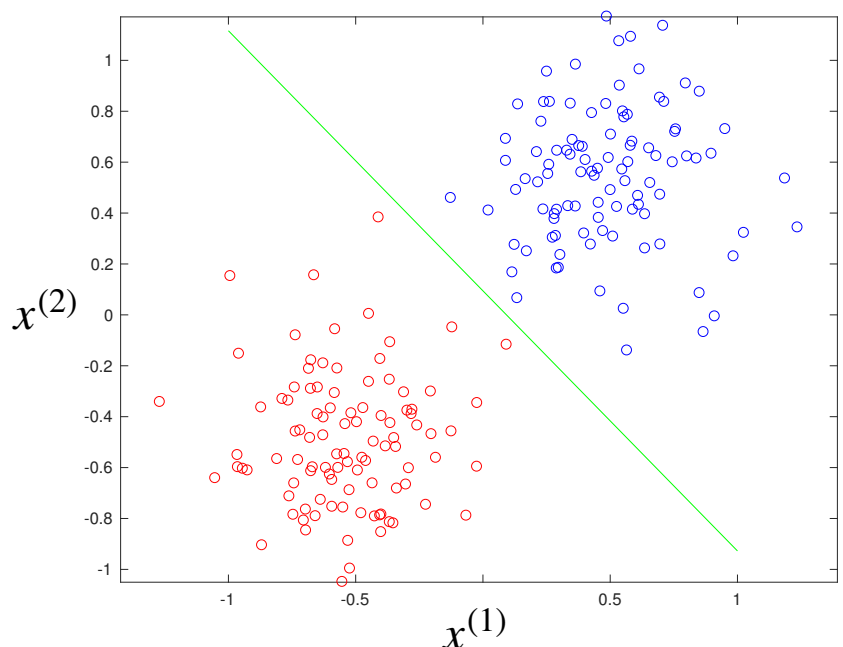


Recap: Decision boundaries

- Different classifiers have different decision boundaries



Least-square classification

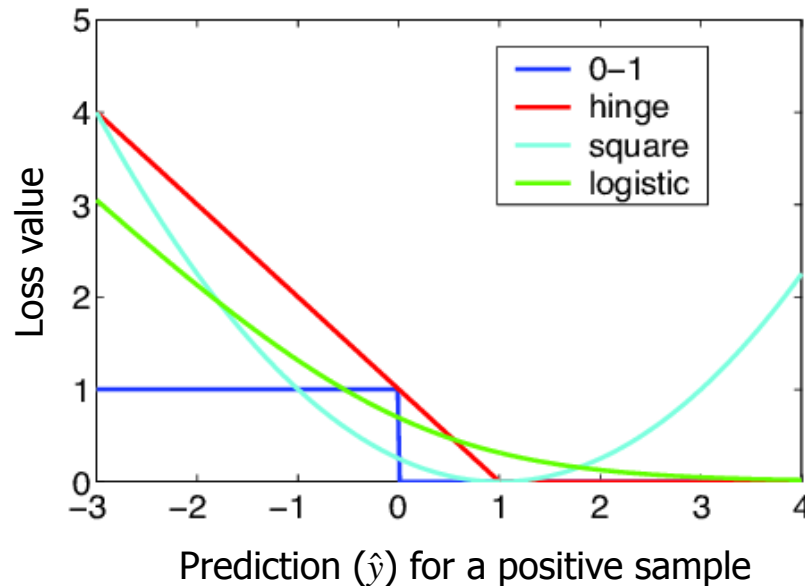


Logistic regression

- Yet they rely on the same linear model
 - So why do they differ?

Recap: Linear models

- The real difference between these methods is the loss function

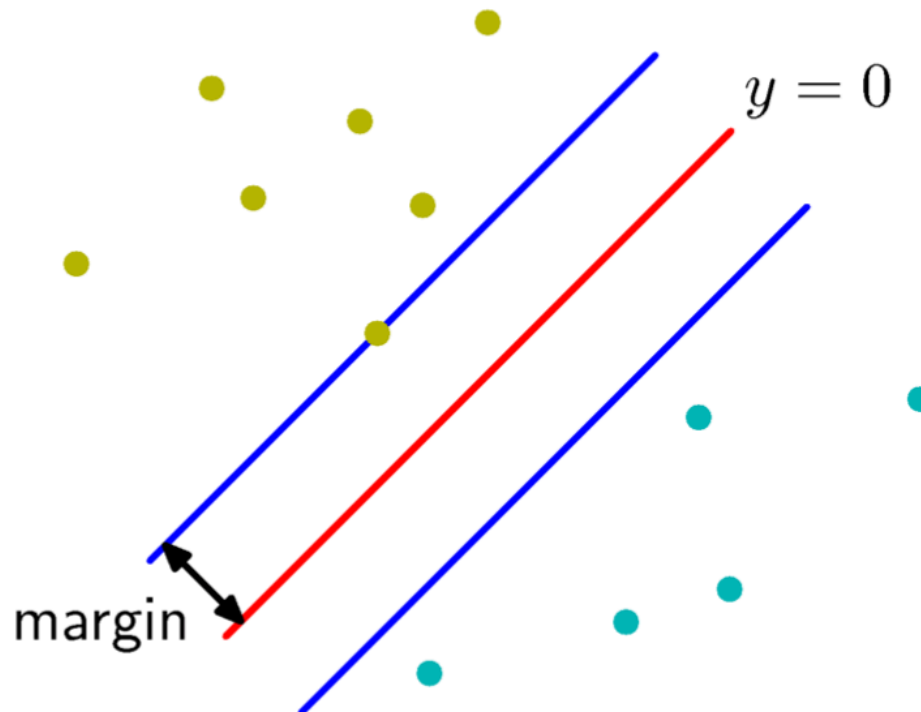


- Square: Least-square classification
- Logistic: Logistic regression
- Hinge: SVM
- 0-1: Ideal

- The choice of loss function has an impact on the resulting algorithm
- Let's have a look at the intuition behind SVM
 - This year I will spare you the mathematical details of SVM

Recap: Margin (Chapter 7.1 in Bishop's book)

- The distance between the decision boundary and the nearest sample is called *margin*
 - Here, the decision boundary is taken at $\hat{y} = 0$, not 0.5 (labels are -1 and 1)



Recap: Support Vector Machine

- The SVM classifier is a binary classifier based on the linear model as before:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

- It aims to maximize the margin while enforcing a correct classification for each training samples
- This is formulated as a convex optimization problem that can be solved to optimality with standard iterative solvers

Recap: SVM: Loss function

- Ultimately, training an SVM can be written as

$$\min_{w^{(0)}, \tilde{\mathbf{w}}} \frac{1}{2C} \|\tilde{\mathbf{w}}\|^2 + \sum_{i=1}^N \max(0, 1 - y_i \cdot (w^{(0)} + \tilde{\mathbf{w}}^T \mathbf{x}_i))$$

- The term

$$\max(0, 1 - y_i \cdot (w^{(0)} + \tilde{\mathbf{w}}^T \mathbf{x}_i))$$

is referred to as the *hinge loss*

Goals of today's lecture

- Define metrics to evaluate a classifier
- Move from binary to multi-class classification problems
- Understand how to represent multiple classes
- Extend the formulation of the linear classifiers seen so far to the multi-class scenario

Recap: Model evaluation

- Once an ML model is trained, one would typically understand how well it performs on unseen test data
 - At this stage, the parameters of the model are fixed
 - Recall that the training and testing data must be separated!
- During this evaluation, one compares the predictions of the model with the true annotations of the test data
 - In contrast to the training stage, the model parameters are *not* updated
 - The evaluation metric may directly be the loss function, but may also differ from it

Classification evaluation

- Confusion matrix: Binary case (e.g., spam vs non-spam email)

True class → Hypothesized class V	Pos	Neg
Yes	TP	FP
No	FN	TN
	$P=TP+FN$	$N=FP+TN$

TP: True positive (number of samples correctly classified as positive)

FP: False positive (number of samples incorrectly classified as positive)

TN: True negative (number of samples correctly classified as negative)

FN: False negative (number of samples incorrectly classified as negative)

P: Total number of positive samples

N: Total number of negative samples

Classification evaluation

- Confusion matrix: Binary case
 - Numerical example for a dataset with 500 positive and 500 negative samples

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

Classification metrics

- Accuracy: Percentage of correctly classified samples

$$Acc = \frac{TP + TN}{P + N}$$

- Example:

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

$$Acc = \frac{200 + 400}{500 + 500} = 0.6$$

What's wrong with accuracy?

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

True class →	Pos	Neg
Yes	400	300
No	100	200
	P=500	N=500

- The two classifiers (matrices) above yield the same accuracy
- But their behavior is very different
 - Left: Weak positive recognition rate, but strong negative recognition rate
 - Right: Strong positive recognition rate, but weak negative recognition rate

Classification metrics

- Precision: Percentage of samples classified as positives that are truly positives

$$Precision = \frac{TP}{TP + FP}$$

- Example:

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

$$Precision = \frac{200}{200 + 100} = 0.667$$

Classification metrics

- Recall: Percentage of positive samples that are correctly classified as positives

$$Recall = \frac{TP}{P}$$

- Example:

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

$$Recall = \frac{200}{200 + 300} = 0.4$$

What's wrong with precision/recall?

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

True class →	Pos	Neg
Yes	200	100
No	300	0
	P=500	N=100

- The two classifiers (matrices) above yield the same precision and recall (0.667 and 0.4) (The datasets are different)
- But their behavior is very different
 - Same positive recognition rate
 - Very different negative recognition rates (strong on left, but nil on right)
- Accuracy would clearly show this

Classification metrics

- False positive rate: Percentage of negative samples that are incorrectly classified as positives

$$FP\ rate = \frac{FP}{N}$$

- Example:

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

$$FP\ rate = \frac{100}{500} = 0.2$$

Classification metrics

- F1 score: Single number that combines precision and recall

$$F1 = 2 \cdot \frac{\textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

- Example:

True class →	Pos	Neg
Yes	200	100
No	300	400
	P=500	N=500

$$F1 = 2 \cdot \frac{0.667 \cdot 0.4}{0.667 + 0.4} = 0.5$$

ROC curve

- Many classifiers output a score/confidence for their predictions (e.g., $\hat{y}$ is a continuous value between 0 and 1 in logistic regression)
- The decision between positive and negative can be computed by thresholding this score:

$$\text{label}_i = 1 \text{ if } \hat{y}_i \geq \tau$$

where τ is a given threshold

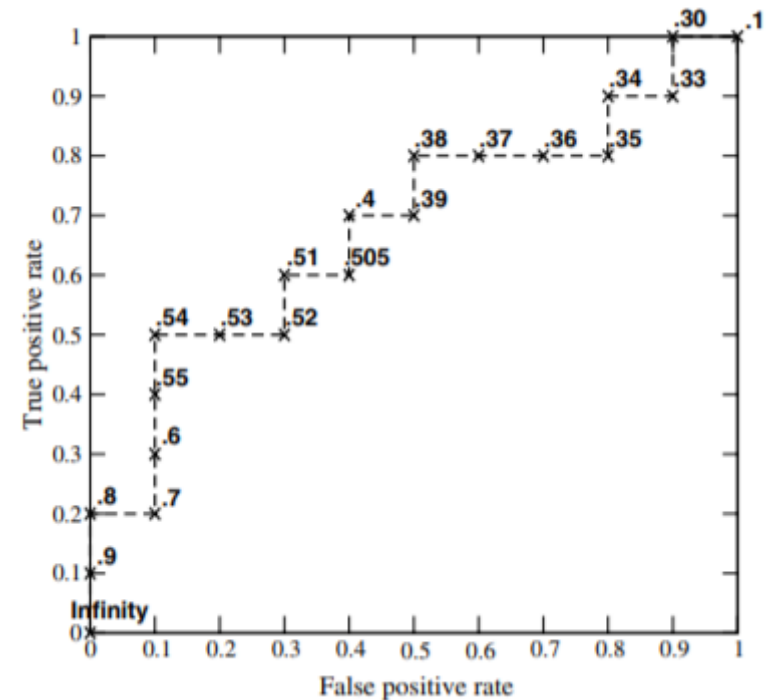
- The Receiver Operating Characteristic (ROC) curve plots the true positive rate (recall) as a function of the false positive rate, obtained by varying the score threshold

Constructing an ROC curve

- Example with 10 positive and 10 negative samples

Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1

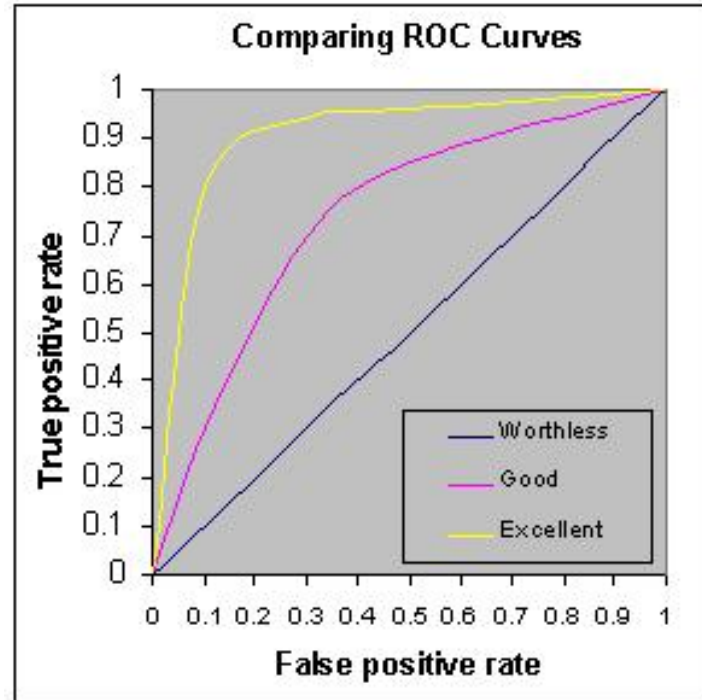
Samples sorted by score
(Class denotes the true label)



ROC curve

Classification metrics

- ROC curves



- The Area Under the ROC Curve (AUC) can act as a metric
 - The maximum AUC is 1
 - Random predictions give and AUC of 0.5

From binary to multi-class classification

- So far, the three methods we have studied can only handle two classes: positive vs negative
- In general, one would typically like to discriminate between more than two categories. E.g., for image recognition



Label 1



Label 2

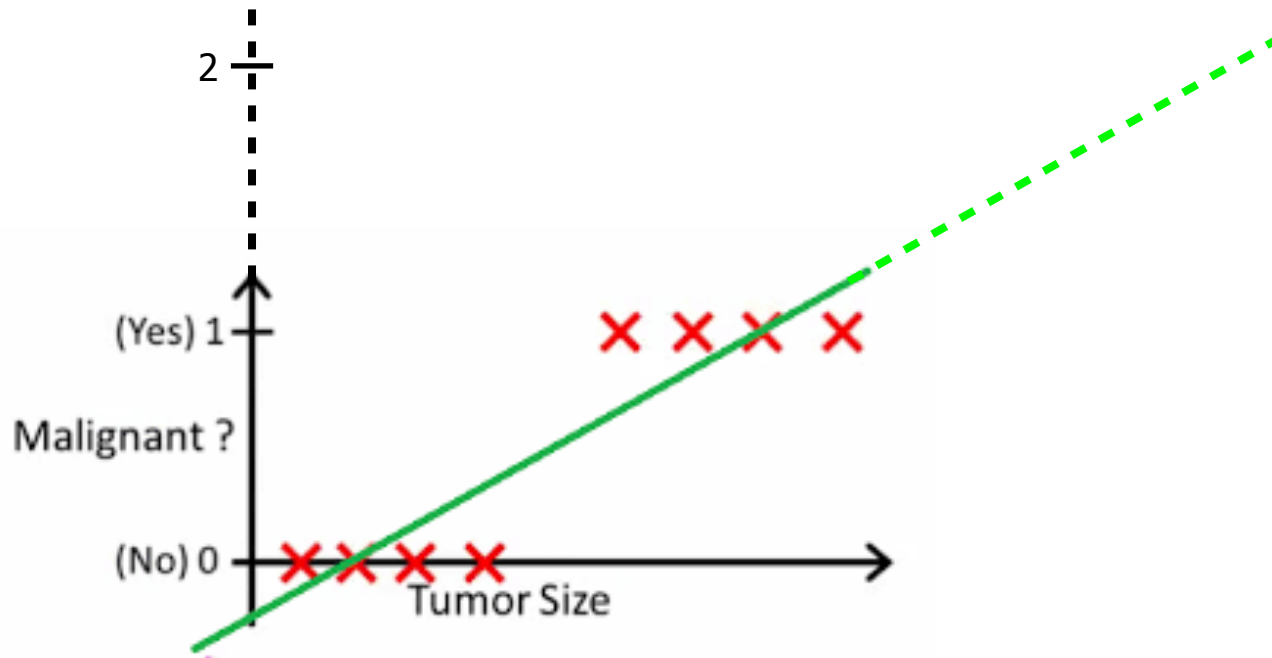


Label 3

- Let us now see how our three linear methods can handle this scenario
 - We will start with least-square classification

Least-square classification

- In essence, least-square classification treats the problem as a regression task
 - The model predicts a continuous value $\hat{y} = w^{(0)} + w^{(1)}x$
 - There is nothing that explicitly makes this value stop at 1, so we could continue the line further



Multi-class least-square classification: Naive approach

- The simplest way to go from the binary case to handling multiple classes would then be to use one integer value for each category



Label 1



Label 2



Label 3

- And round the prediction of the model to the nearest integer to obtain the predicted class value
- The same framework as for least-square binary classification would then directly apply

Multi-class least-square classification: Naive approach

- This, however, makes no sense, because:
 - There is no natural order between the different categories
 - E.g., mistaking a car for a tree would be less strongly penalized than mistaking it for a cat
 - This is because the least-square loss would be $(3 - 2)^2 = 1$ in the first case and $(3 - 1)^2 = 4$ in the second



Label 1



Label 2



Label 3

Dealing with multiple classes

- The most common approach to modeling a multi-class problem consists of encoding the class label as a vector with a single 1 at the index corresponding to the category and 0s elsewhere
 - In a 5-class problem, a sample in class 2 is represented as

$$\mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- This is referred to as *one-hot encoding* (or 1-of-C encoding)

Multi-output linear model

- Predicting such a one-hot encoding is then similar to making the model output multiple values, as we saw for multi-output regression
- We therefore again use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1) \times C}$, such that

$$\hat{\mathbf{y}}_i = \mathbf{W}^T \mathbf{x}_i = \begin{bmatrix} \mathbf{w}_{(1)}^T \\ \mathbf{w}_{(2)}^T \\ \vdots \\ \mathbf{w}_{(C)}^T \end{bmatrix} \mathbf{x}_i$$

where each $\mathbf{w}_{(j)}$ is a $(D + 1)$ -dimensional vector, used to predict one output dimension, i.e., the score for one class

Multi-class least-square classification

- Multi-class classification is then similar to a multi-output regression problem
- We can then write training as

$$\min_{\mathbf{W}} \sum_{i=1}^N \|\mathbf{W}^T \mathbf{x}_i - \mathbf{y}_i\|^2$$

where each $\mathbf{y}_i$ is now a C -dimensional vector with a single one at the index corresponding to the class label, and zeros everywhere else

Multi-class least-square classification: Solution

- We can again group the inputs and the outputs in two matrices, as

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}_1^T \\ \mathbf{y}_2^T \\ \vdots \\ \mathbf{y}_N^T \end{bmatrix} \in \mathbb{R}^{N \times C}$$

- Then, the solution has the same form as in the regression case (obtained by zeroing out the gradient w.r.t. $\mathbf{W}$), i.e.,

$$\mathbf{W}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} = \mathbf{X}^\dagger \mathbf{Y}$$

Multi-class least-square classification

- Given the optimal parameters, the prediction for a new sample is computed as

$$\hat{\mathbf{y}} = (\mathbf{W}^*)^T \mathbf{x}$$

- The sample is then assigned to class k if

$$\hat{\mathbf{y}}^{(k)} > \hat{\mathbf{y}}^{(j)} , \quad \forall j \neq k$$

- Or in other words the final label k is obtained as

$$k = \operatorname{argmax}_j \hat{\mathbf{y}}^{(j)}$$

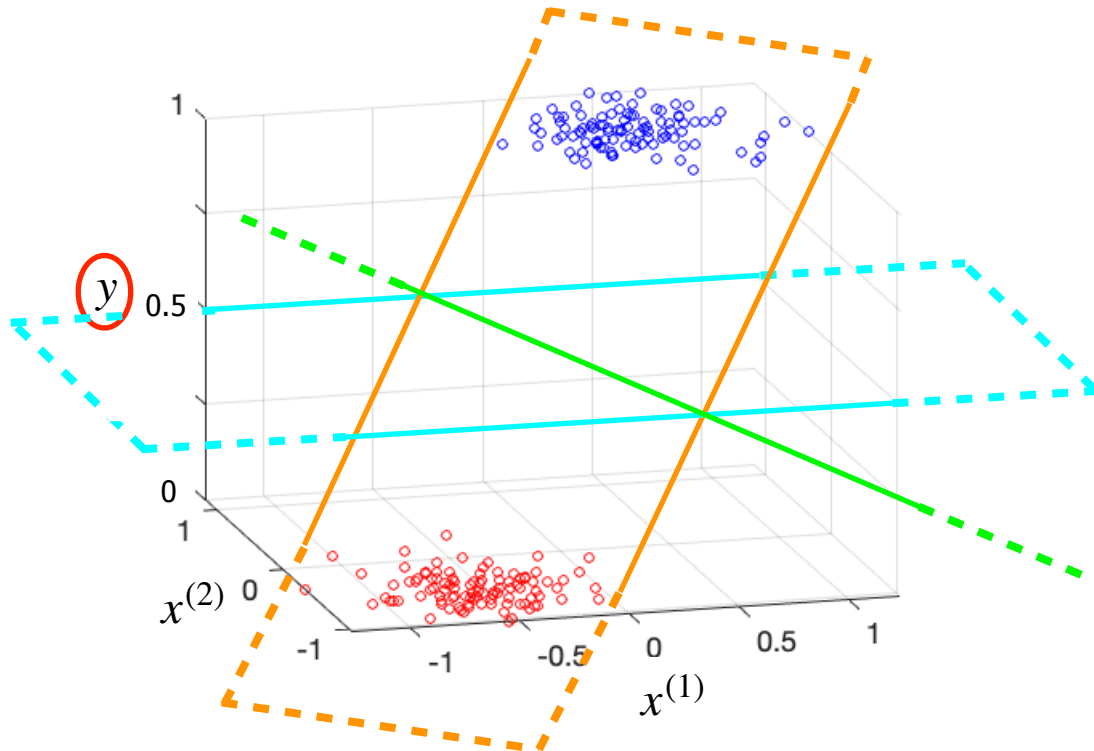
- That is, the label is the index (position) of the maximum value in the vector $\hat{\mathbf{y}}$

Decision Boundary

(The following applies to all linear classification models, not just least-square classification)

Recap: Decision boundary

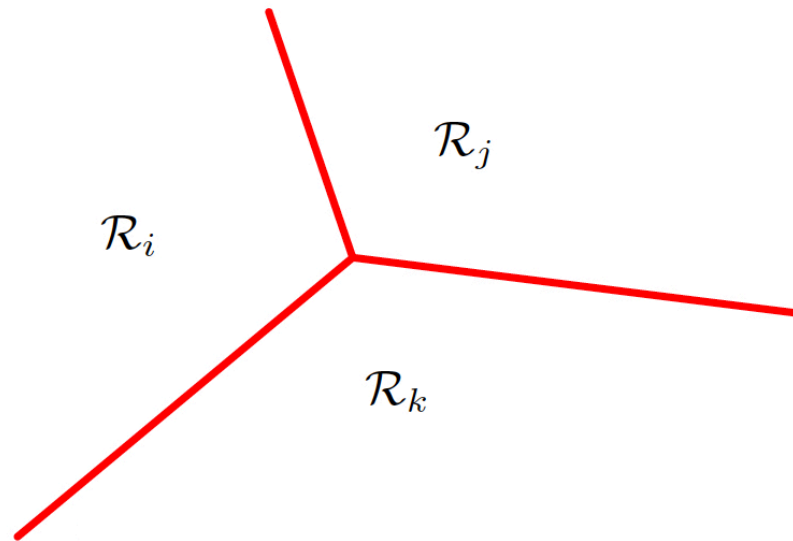
- In the binary case, the point where the predicted label changes from 0 to 1 forms a *decision boundary*
 - With 2D inputs, it is a line
 - To understand this, let's visualize the data in 3D (with y as 3rd dimension)



- The linear model encodes the plane that passes through the samples (orange plane)
- The decision boundary (green line) is the intersection of this plane with the plane $y = 0.5$ (cyan plane)

Decision boundaries: General definition

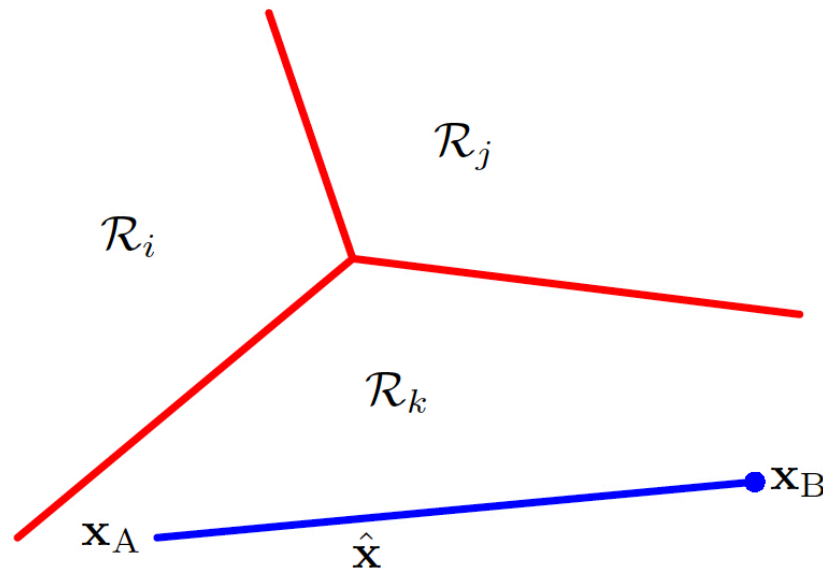
- The decision boundary between two classes k and j is defined as the set of points $\mathbf{x}$ such that $\hat{y}^{(k)}(\mathbf{x}) = \hat{y}^{(j)}(\mathbf{x})$
- This corresponds to the $(D - 1)$ -dimensional hyperplane



- The binary case is a special case of this definition

Decision boundaries

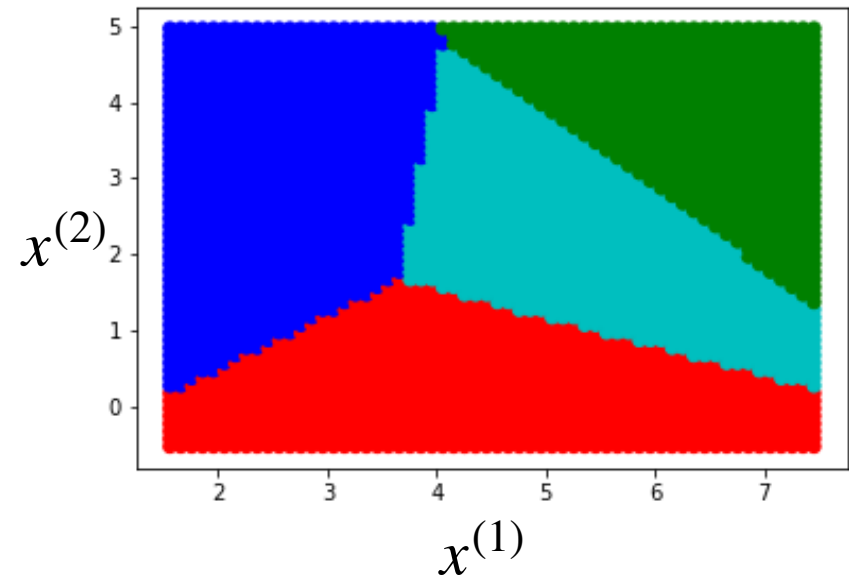
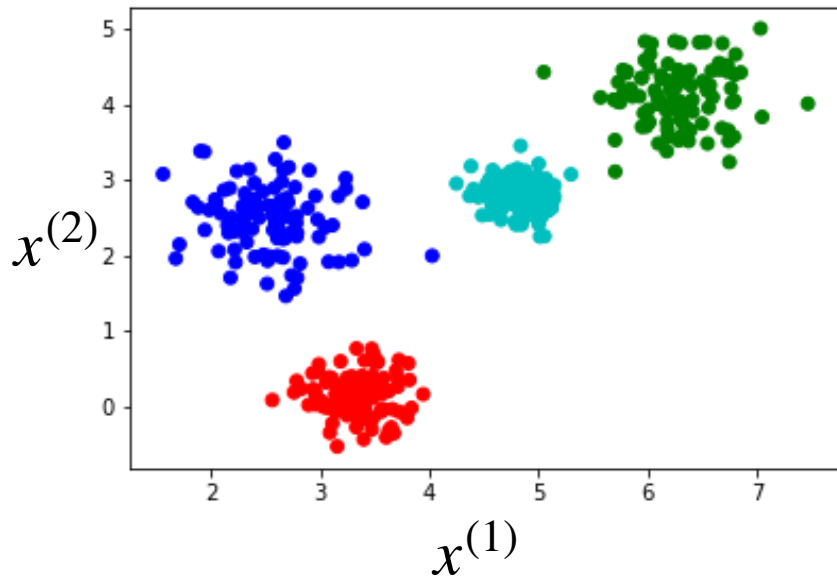
- The decision regions are singly-connected and convex



- Any point $\hat{\mathbf{x}}$ between two points $\mathbf{x}_A$ and $\mathbf{x}_B$ in the same region (e.g., k) will also be in the same region

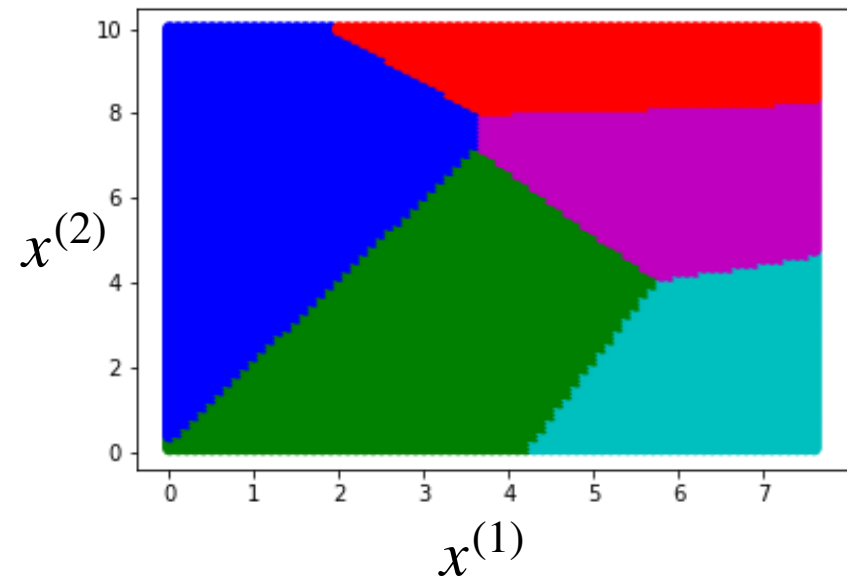
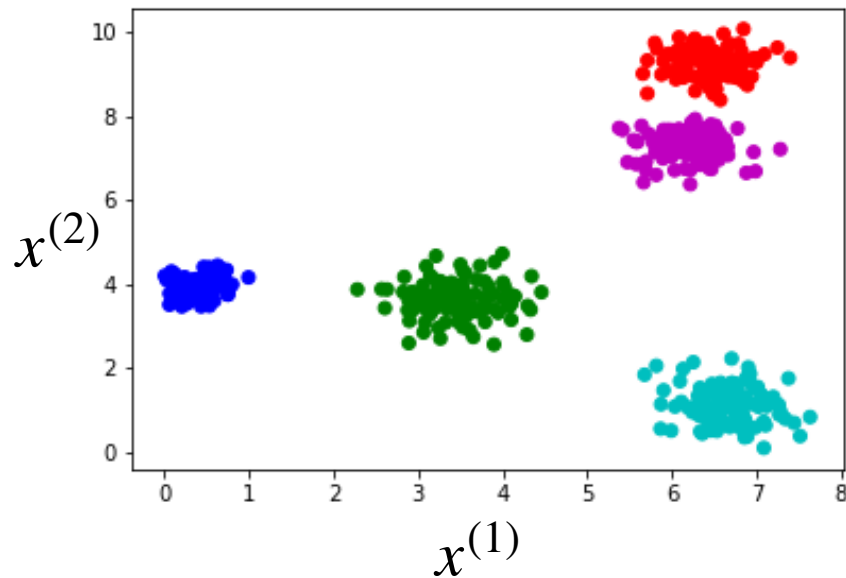
Decision boundaries for $C > 3$ classes

- $C = 4$ classes



Decision boundaries for $C > 3$ classes

- $C = 5$ classes

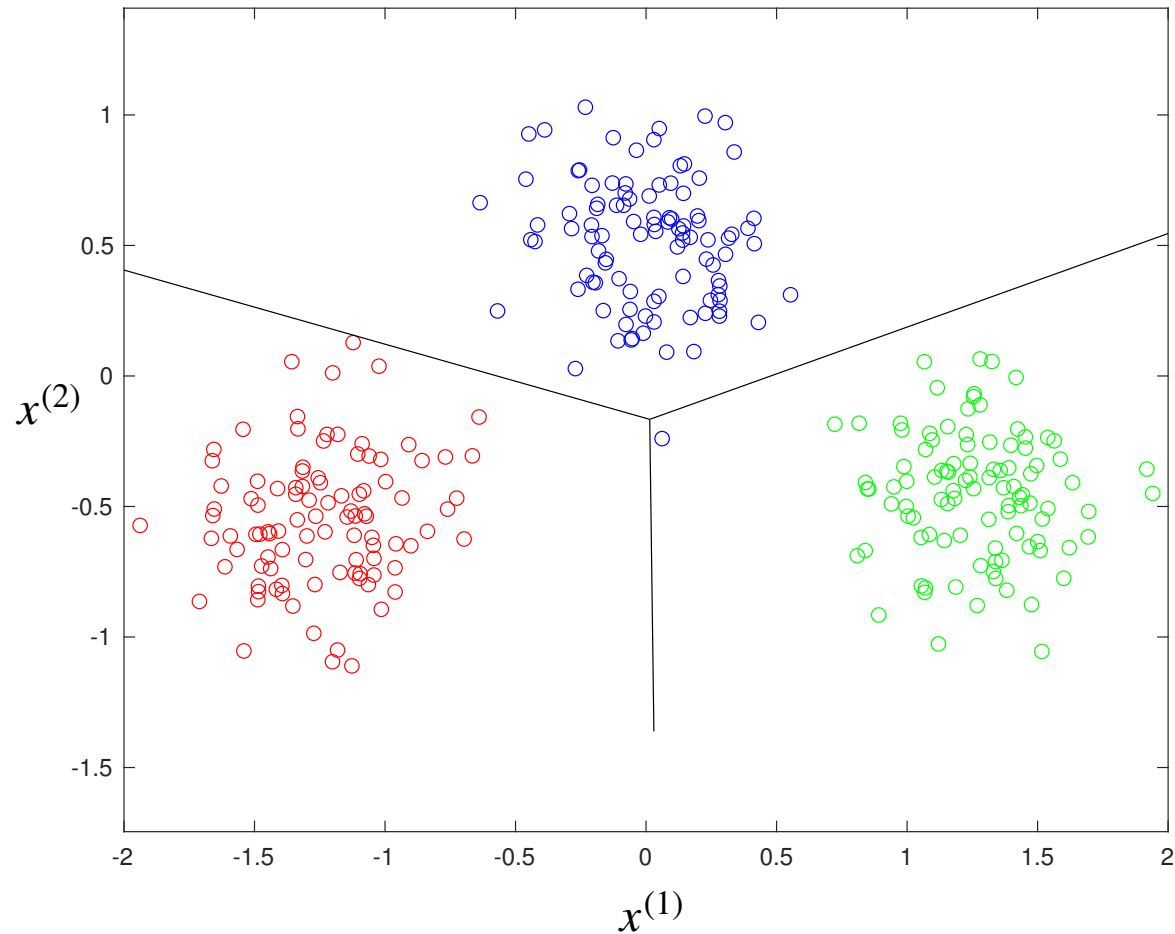


End of the interlude

Let's look at a few least-square
classification examples

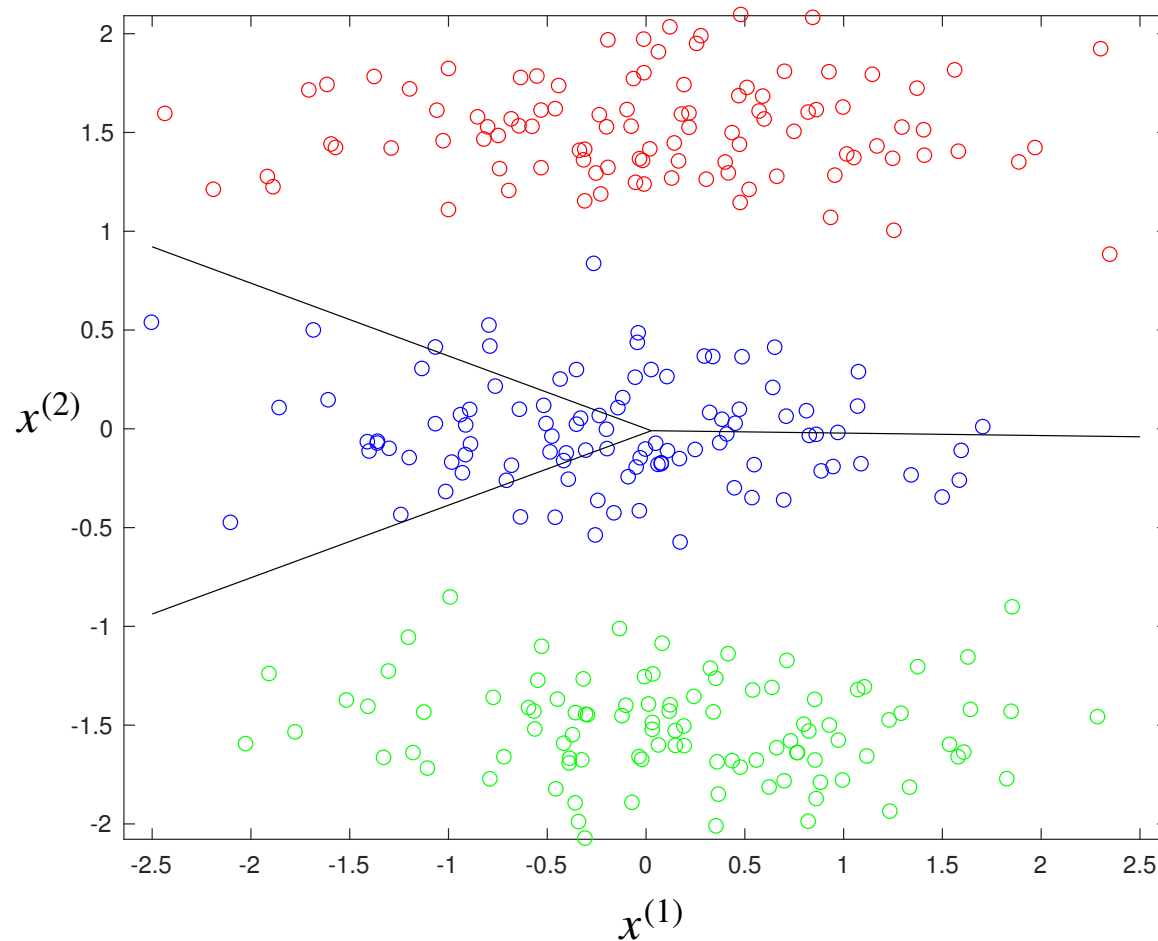
Multi-class least-square classification

- Example: 2D inputs, 3 classes



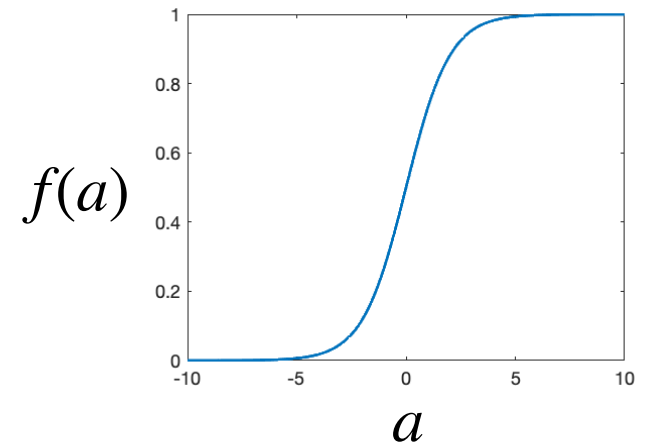
Multi-class least-square classification

- Failure case



Multi-class least-square classification: Drawbacks

- As in the binary case, multi-class least-square classification suffers from the fact that the loss function does not reflect the underlying classification task
- In the binary case, we addressed this by adding a nonlinearity to the output of the linear model via the logistic function, leading to logistic regression
- Let us now see how this extends to the multi-class scenario



Multi-class logistic regression

- As in the least-square classification case, we can use one-hot encodings to represent multi-class labels
- Then, we again need to use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1) \times C}$ to represent the model parameters
- In this case, the probability for a class k is given by the *softmax* function

$$\hat{y}^{(k)}(\mathbf{x}) = \frac{\exp(\mathbf{w}_{(k)}^T \mathbf{x})}{\sum_{j=1}^C \exp(\mathbf{w}_{(j)}^T \mathbf{x})}$$

Multi-class logistic regression

- The empirical risk can then be derived from the multi-class version of the cross entropy, which yields

$$R(\mathbf{W}) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$$

- As in the binary case, a single $y_i^{(k)}$ is 1 for every sample i , so we really have one term per training sample
- As in the binary case, training is done by minimizing this empirical risk using gradient descent

Recap: Gradient descent

- Algorithm:
 1. Initialize $\mathbf{w}_0$ (e.g., randomly)
 2. While not converged
 - 2.1. Update $\mathbf{w}_k \leftarrow \mathbf{w}_{k-1} - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_{k-1})$
- η is a single value referred to as the learning rate
- Convergence can again be measured by
 - Thresholding the change in function value
 - Thresholding the change in parameters, e.g., $\|\mathbf{w}_{k-1} - \mathbf{w}_k\| < \delta_w$

Gradient descent: Dealing with a matrix

- With multiple classes, the parameters are shaped as a matrix $\mathbf{W} \in \mathbb{R}^{(D+1) \times C}$, not as a vector anymore
- This does not affect the algorithm:
 - It just means that the gradient itself will also be a matrix, i.e.

$$\nabla_{\mathbf{W}} R(\mathbf{W}_{k-1}) \in \mathbb{R}^{(D+1) \times C}$$

- So the whole algorithm works with matrix entities
 - Convergence check can be done based on the Frobenius norm:
 $\|\mathbf{W}_{k-1} - \mathbf{W}_k\|_F < \delta_w$

Multi-class logistic regression: Gradient

- By following the chain rule and using the gradient of the softmax function, we can derive the gradient of the multi-class cross-entropy as (details in Bishop Chap. 4.3.4)

$$\nabla_{\mathbf{W}} R(\mathbf{W}) = \sum_{i=1}^N \mathbf{x}_i (\hat{\mathbf{y}}_i - \mathbf{y}_i)^T$$

- The gradient follows the same intuition as in the binary case: It multiplies the input by the error of the current prediction for each class $\left(\hat{y}_i^{(k)} - y_i^{(k)} \right)$

Multi-class logistic regression: Prediction

- At test time, given a new input sample $\mathbf{x}$, the probability/score for any class k is computed via the softmax function as

$$\hat{y}^{(k)}(\mathbf{x}) = \frac{\exp((\mathbf{w}_{(k)}^*)^T \mathbf{x})}{\sum_{j=1}^C \exp((\mathbf{w}_{(j)}^*)^T \mathbf{x})}$$

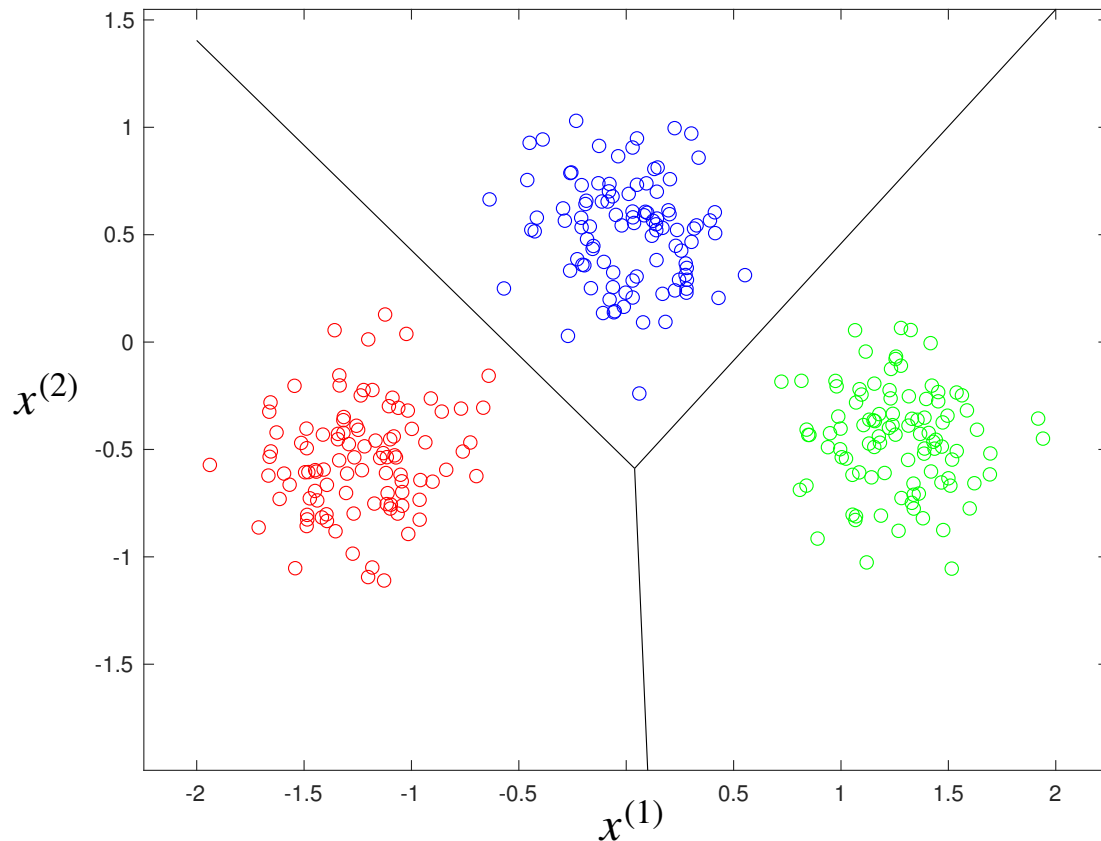
- The final class label is then predicted as

$$k = \operatorname{argmax}_j \hat{y}^{(j)}(\mathbf{x})$$

- That is, as in least-square classification, the label is the index (position) of the maximum value in the vector $\hat{\mathbf{y}}$

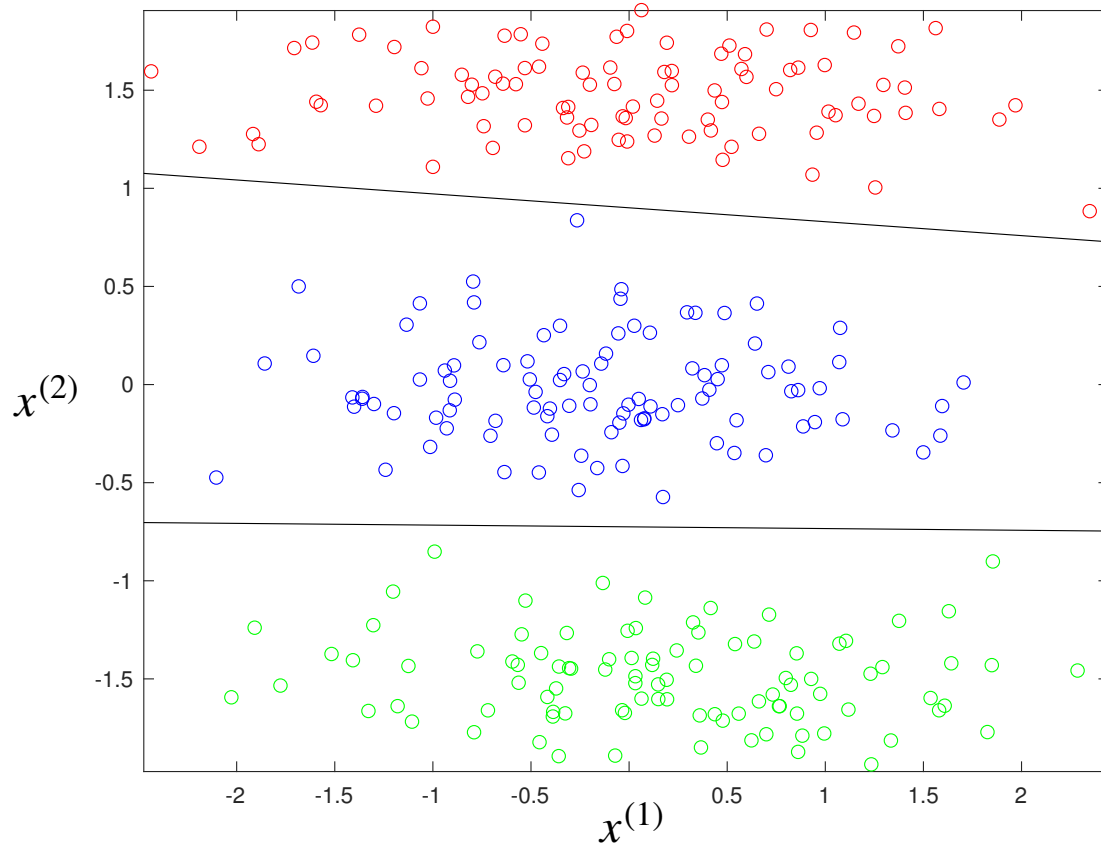
Multi-class logistic regression: Example

- 2D inputs, 3 classes: Logistic regression results



Multi-class logistic regression

- 2D inputs, 3 classes: Logistic regression results



Evaluation metrics for multi-class classification

Classification evaluation

- Confusion matrix: Multi-class case
 - Numerical example for a dataset with 3 classes (UCI Iris dataset)
 - Note that the matrix is transposed (predicted labels as columns and actual labels as rows), which can also be used in practice

		Predicted		
		<i>Iris-setosa</i>	<i>Iris-versicolor</i>	<i>Iris-virginica</i>
Actual	<i>Iris-setosa</i>	12	1	1
	<i>Iris-versicolor</i>	0	16	0
	<i>Iris-virginica</i>	0	1	16

Metrics for multi class problems

- Accuracy:
 - A global accuracy can be obtained by summing the correct predictions for all classes (diagonal values of the confusion matrix), and dividing this by the total number of samples (sum of all entries in the confusion matrix)
- Precision, recall, FP-rate and F1-score:
 - The simplest approach to handling these metrics is to compute them for each individual class, and then average over the classes
 - In this case, each individual class is in turn considered as the “positive” class while the others act as the “negative” class
 - This does not account for class imbalance (some classes may have many more samples than others; more about this in a future lecture)
 - This can be handled by weighting the metric for each class by a value depending on the proportion of samples from that class

End of the interlude

Let's look at a practical multi-class
logistic regression example

Multi-class logistic regression: Application

- Predict the state of a fetus from cardiotocography: UCI Cardiotocography Dataset

- 21 input features
- Predict fetal state as Normal vs Suspect vs Pathologic

S. number	Name of the features	Description
1	LB	FHR baseline (beats per minute)
2	AC	Number of accelerations per second
3	FM	Number of fetal movements per second
4	UC	Number of uterine contractions per second
5	DL	Number of light decelerations per second
6	DS	Number of severe decelerations per second
7	DP	Number of prolonged decelerations per second
8	ASTV	Percentage of time with abnormal short term variability
9	MSTV	Mean value of short term variability
10	ALTV	Percentage of time with abnormal long term variability
11	MLTV	Mean value of long term variability
12	Width	Width of FHR histogram
13	Min	Minimum of FHR histogram
14	Max	Maximum of FHR histogram
15	Nmax	Number of histogram peaks
16	Nzeros	Number of histogram zeros
17	Mode	Histogram mode
18	Mean	Histogram mean
19	Median	Histogram median
20	Variance	Histogram variance
21	Tendency	Histogram tendency: -1 = left asymmetric;
		0 = symmetric; 1 = right asymmetric

Multi-class logistic regression: Application

- Predict the state of a fetus from cardiotocography: UCI Cardiotocography Dataset

- Logistic regression yields an accuracy of 92.1%

- Confusion matrix

Predicted label

	404	2	4
True label	24	48	0
	2	10	38

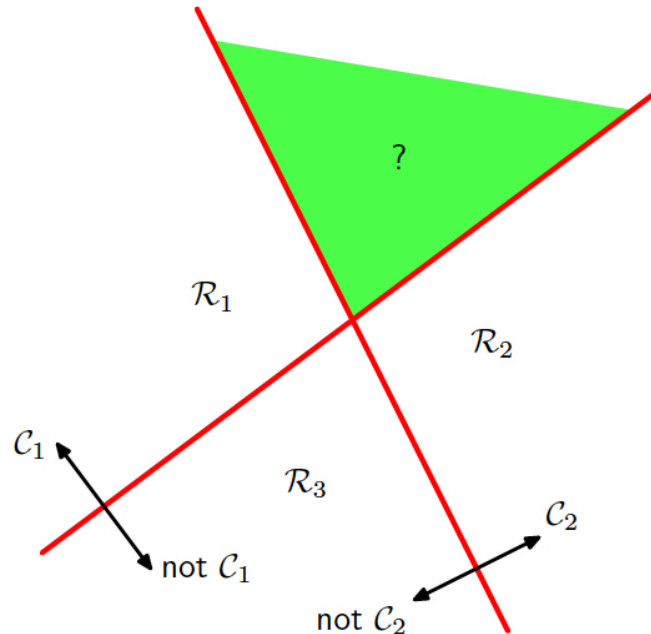
- Still 26 suspicious/pathologic samples classified as normal!

Dealing with multiple classes

- So far, we have seen how to extend binary least-square classification and binary logistic regression to the multi-class scenario. How about SVM?
- Unfortunately, there is no ideal solution to handle multiple classes in SVM
- Instead, one typically relies on one of the two solutions described in the next slides
 - Note that these solutions are quite general, not specific to SVM

Dealing with multiple classes

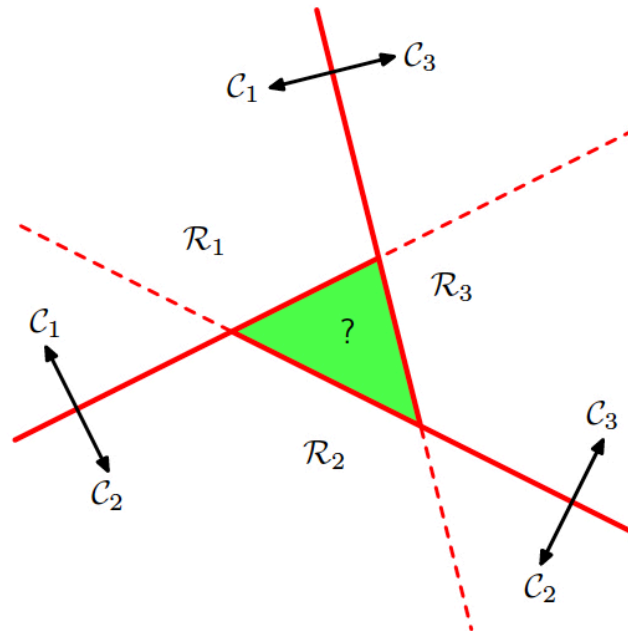
- To deal with multiple classes, one can use several binary classifiers
 - E.g., one-vs-rest: Classify each individual category vs all the other ones jointly



- This, however, leaves ambiguities (e.g., a sample can be classified as being from both class 1 and class 2)

Dealing with multiple classes

- To deal with multiple classes, one can use several binary classifiers
 - E.g., one-vs-one: Train one classifier for every pair of classes



- This also may result in inconsistencies (e.g., a sample can be classified as being from class 1, class 2 and class 3)

Comparing linear classifiers

- We have now seen 3 different linear (multi-class) classifiers:
 - Least-square classification
 - Logistic regression
 - SVM
- In practice, one almost never uses the least-square one
 - It does not reflect the underlying classification task, but is useful from a pedagogical standpoint
- Let us now compare logistic regression and SVM
 - The following results come from <https://martin-thoma.com/comparing-classifiers/>

Comparison on the Iris dataset

- UCI Iris dataset
 - 150 samples
 - 4 attributes (petal and sepal length and width)
 - 3 iris species (i.e., 3 classes)

	Accuracy	Training time	Testing time
SVM, linear	88.00%	0.0006s	0.0002s
Logistic Regression (C=1)	88.00%	0.0011s	0.0001s
Logistic Regression (C=1000)	92.00%	0.0010s	0.0002s

The value C is a hyper-parameter encoding “regularization” strength. We will talk about this in a future lecture; let’s ignore it for now

- These results may suggest that logistic regression is better, however...

Comparison on the MNIST dataset

- MNIST handwritten digit recognition dataset
 - 60'000 training samples, 10'000 test samples
 - 28×28 images
 - 10 digits: 0,...,9 (i.e., 10 classes)

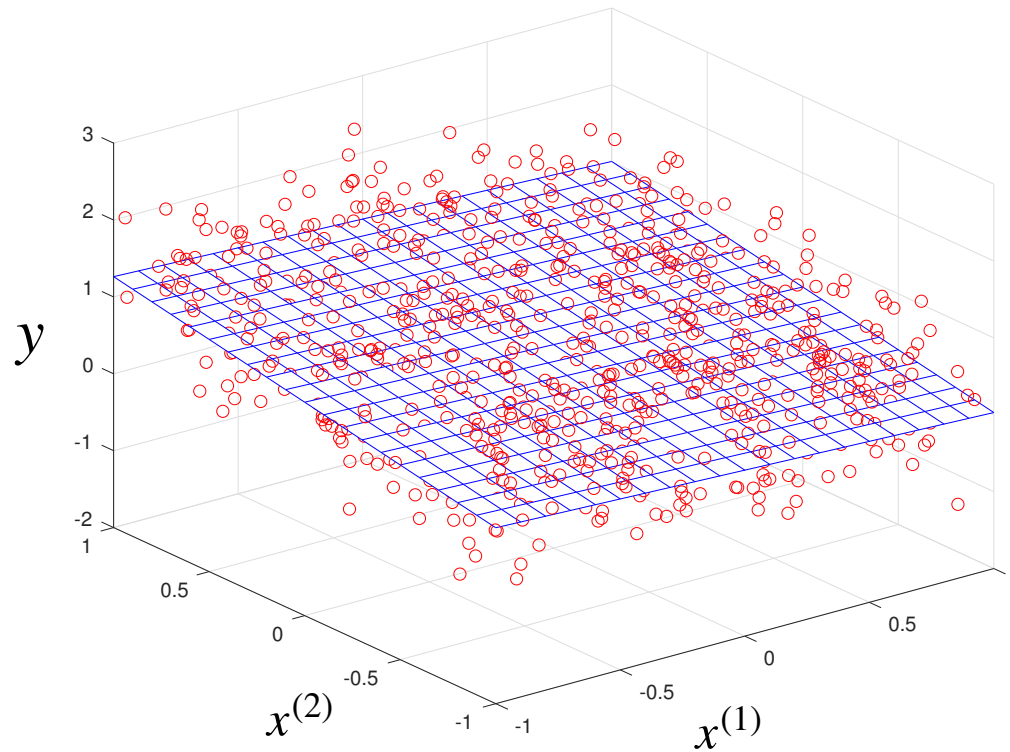
	Accuracy	Training time	Testing time
Linear SVM	94.16%	168.6950s	158.0101s
Logistic Regression (C=1)	91.47%	272.1309s	0.0531s
Logistic Regression (C=10000)	91.23%	1807.0624s	0.0529s

The value C is a hyper-parameter encoding “regularization” strength. We will talk about this in a future lecture; let’s ignore it for now

- The choice of the best classifier depends on the data

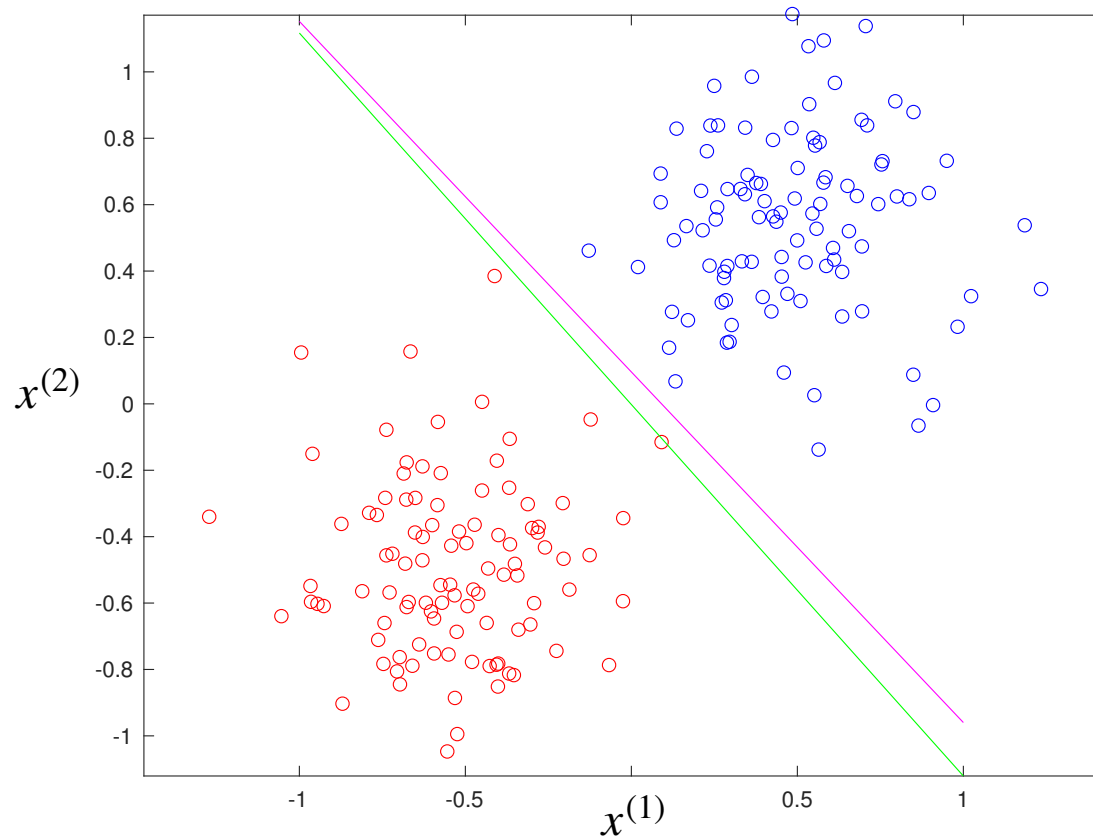
Until now

- Linear regression: Fit a linear model to the observations



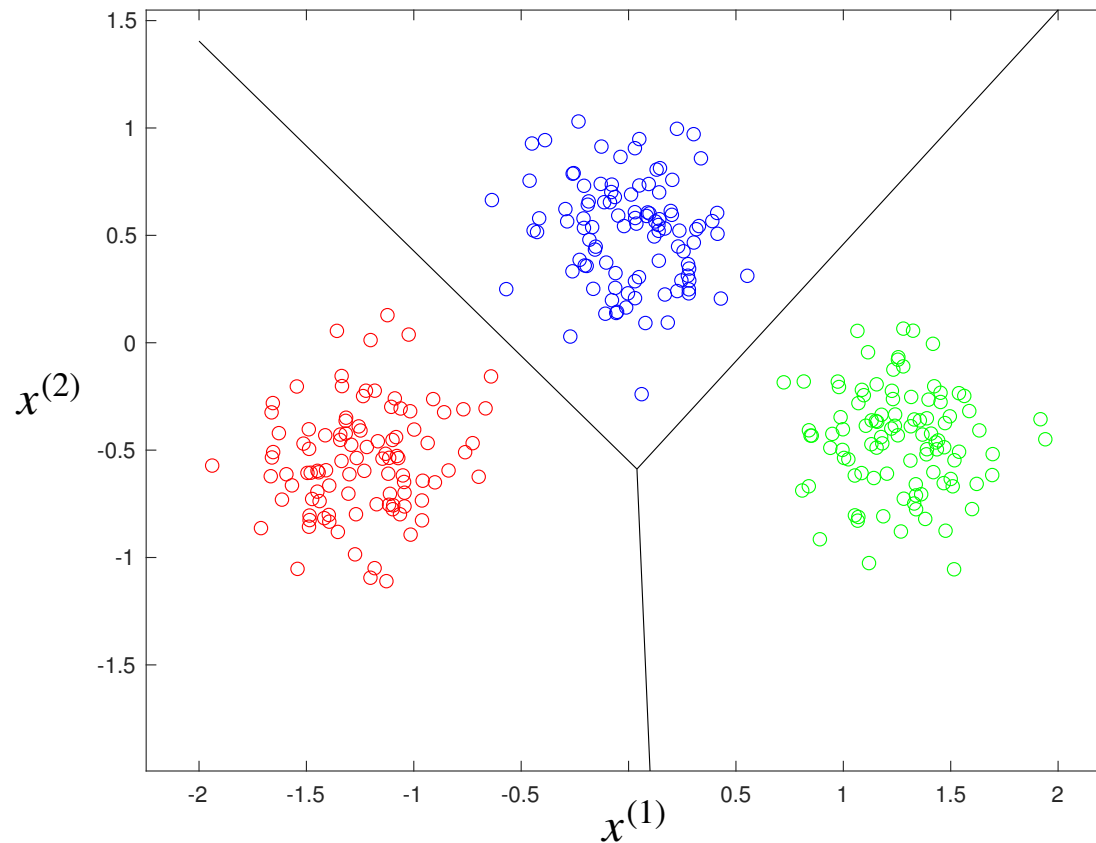
Until now

- Binary linear classification: Use a linear model to separate the positive and negative samples



Until now

- Multi-class linear classification: Use a linear model to separate the classes, via a one-hot encoding of the label



From linear to nonlinear

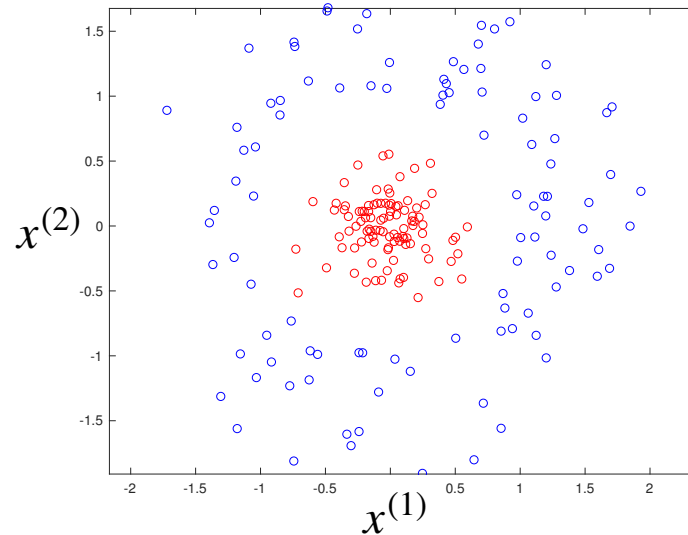
- Unfortunately, no linear model can directly fit this data

- 1D input, 2 classes (colors)



- Or this data

- 2D input, 2 classes (colors)



From linear to nonlinear

- Or this even more complicated data

